CASE STUDY

FanPulse

Real-time AI fan sentiment dashboard for the 2026 FIFA World Cup

Liwash Saikia

MSc Management, Queen's University Belfast

fanpulse-blush.vercel.app

github.com/theliwash/fanpulse

Overview

FanPulse is a full-stack web application that tracks live World Cup match data and uses artificial intelligence to generate real-time fan sentiment analysis per nation. The product surfaces the emotional context around match events, classifying fan mood as Euphoric, Confident, Nervous, Frustrated, Devastated, or Furious, and presents this through an interactive dashboard, individual nation pages with historical timelines, and a pre-match AI prediction feature that compares predicted sentiment against post-match reality.

The application has been running continuously since June 11, 2026, the opening day of the FIFA World Cup.

Motivation

This project was not initiated to address a specific market gap. The primary objective was to develop practical competency in full-stack product development by building, shipping, and iterating on a live product under real constraints. The hypothesis was straightforward: the most effective learning occurs when a product breaks in production and must be diagnosed and resolved under pressure.

Product Scope and Decision Making

The initial product concept centred on one specific value proposition: surfacing the emotional dimension of football, not just the statistical one. Scores, fixtures, and standings are available everywhere. What FanPulse set out to provide was the sentiment layer on top of that data.

During development, the feature set began to expand toward a broader football data product, incorporating richer fixture data, more detailed match statistics, and a more comprehensive results experience. This represented a meaningful scope risk. The decision was made to pull the product back to its original intent. Every feature considered but not built was a deliberate product decision, not an oversight. Scope discipline was the most important product management practice exercised throughout this project.

Technical Architecture

Frontend and API layer: Next.js 14 with TypeScript and Tailwind CSS, deployed on Vercel. All backend logic is handled through Next.js API routes, eliminating the need for a separate server.

Database: Supabase (PostgreSQL) with row-level security policies configured for all tables. Three core tables: matches, sentiment snapshots, and user reactions.

AI layer: Groq API with the Llama 3.3 70B Versatile model. A structured prompt returns sentiment as a validated JSON array including mood label, sentiment score from -100 to 100, top emotion, and a key talking point per nation.

Data source: football-data.org for live World Cup fixture data and match scores.

Automation: cron-job.org triggers a sentiment analysis job every 30 minutes. The cron uses time-based match detection rather than API-reported match statuses, which proved unreliable on the free tier.

Total infrastructure cost: Zero pounds.

Key Features

- Dashboard with live, upcoming, and finished match filters. Filter state persists across page refreshes via URL query parameters.

- Nation pages displaying a match timeline chart showing how sentiment evolved during a specific game, and a tournament journey view showing sentiment across all matches.

- AI versus Reality feature: a pre-match prediction is generated before kickoff and compared against actual post-match sentiment, making model accuracy visible to the user.

- Persistent emoji reactions stored in Supabase and incorporated into the AI analysis prompt, providing human context alongside match score data.

- Automated sentiment analysis running every 30 minutes via cron-job.org, processing any match from the preceding 24 hours that does not have a recent sentiment snapshot.

Production Challenges and Resolutions

Supabase insert failures: All sentiment analysis was completing successfully but no data was being persisted. Investigation revealed that server-side API routes were using the public anonymous key rather than the service role key, which was blocked by row-level security policies. The resolution required a single configuration change but required systematic debugging to identify.

Unreliable match status data: Matches were being reported as live when they had finished and as scheduled when they were actively in play. The resolution was to remove dependency on the API-reported status field entirely and implement time-based match detection. If a match kicked off within the preceding 24 hours and has no recent sentiment snapshot, the cron job processes it regardless of what status the API returns.

Stale Vercel deployments: Vercel's GitHub integration was silently deploying older commits rather than the latest code. This was resolved by switching to direct CLI deployment using the Vercel CLI for all production releases.

Cron route static caching: The automated sentiment cron route was being pre-rendered as a static page by Next.js, causing it to return a cached response on every call. Adding the export const dynamic

directive resolved this and restored correct dynamic execution.

Reflections

The primary improvement identified for a future iteration is the visual design. The product is functionally complete but the user interface would benefit from more considered design investment. Given additional time, visual polish would be prioritised earlier in the development cycle rather than addressed retrospectively.

The broader learning from this project is that production systems fail in ways that local development environments do not reveal. Diagnosing and resolving those failures requires a working understanding of the entire system stack, from database access policies to deployment pipelines to API response caching. That systems-level thinking is the primary competency developed through this project.

---

fanpulse-blush.vercel.app